# Föreläsning 14

Dynamisk programmering

# Föreläsning 14

- Kursutvärdering
- Dynamisk programmering
- Fibonacci
- Myntväxling
- Floyd-Warshall
- Kappsäck
- Handelsresandeproblemet

# Kursutvärdering

Vi gör den nu.

# Dynamisk programmering

Dynamisk programmering används typiskt när man vid uppdelning av ett problem i subproblem får subproblem som överlappar. En naiv metod blir då ineffektiv eftersom den löser samma problem flera gånger. Grundtanken med dynamisk programmering är då att se till att vi löser varje subproblem endast en gång.

Top-down: Rekursiv uppdelning där vi memorerar (memoize) subproblem när de är lösta (tabellerar) och alltid kontrollerar om vi redan löst ett problem innan vi löser det.

Bottom-up: Här börjar vi med de minsta subproblemen och bygger i steg upp lösningar på större och större problem. Ibland behöver man även här tabulera resultat och ibland blir det helt enkelt en iterativ lösning.

En del programspråk har automatiskt stöd för memoreringen via funktionsanrop som sparar resultat av tidigare anrop.

# Fibonacci

```
static long fib(int n){
    if(n==1||n==2) return 1;
    return fib(n-1)+fib(n-2);
}
```

-**fib(40) ger 204.668.309 anrop**

**Dynamiskt top-down**

```
static long fibDyn(int n) {
    long[] tab = new long[n+1];
    tab[1] = tab[2] = 1;
    return fibDyn(n, tab);
}

static long fibDyn(int n, long[] tab) {
    if (tab[n] != 0) return tab[n];
    tab[n] = fibDyn(n-1, tab) + fibDyn(n-2, tab);
    return tab[n];
}
```

-**fibDyn(40) ger 77 anrop**

**Dynamiskt bottom-up:**

```
static long fibIterativ(int nFinal)
{
    long fn=1,fnMinus1=1,fnMinus2=1;
    for(int n=3;n<=nFinal;n++){
        fn=fnMinus1+fnMinus2;
        fnMinus2=fnMinus1;
        fnMinus1=fn;
    }
    return fn;
}
```

# Myntväxling

Input: belopp kr i växel

mynt – array med de n olika valörerna

**<u>växel(belopp)</u>**

```
För i = 0 till n-1
        Om mynt[i]=belopp return 1
bäst=∞
För i = 0 till n-1
        bäst=min(bäst,1+växel(belopp-mynt[i]))
return bäst
```

Hur gör vi dynamiskt?

# Myntväxling - dynamiskt

```
För i=0 till n–1
        tab[mynt[i]]=1; (övriga positioner ska vara 0)
```

**<u>växel(belopp)</u>**

```
        Om tab[belopp]≠0 return tab[belopp]
        bäst=100000
        För i = 0 till n-1
                bäst=min(bäst,1+växel(belopp-mynt[i]))
        tab[belopp] = bäst
        return tab[belopp]
```

# Floyd-Warshall

Löser kortaste vägen i en viktad graf.

Hittar kortaste vägen mellan varje par av noder i $O(|V|^3)$ vilket är ganska snabbt med tanke på att det kan finnas $O(|V|^2)$ bågar.

Hittar endast avståndet och inte vilken väg man ska ta.

Kan hantera negativa vikter till skillnad från Dijkstra's.

Använder dynamisk programmering, bottom up, för att åstadkomma detta.

# Princip

Input:
V – mängden av noder numrerade 1, …, N
w(i,j) – vikt för båge från nod i till nod j

kortasteVägen(i,j,k) –
kortaste vägen från nod i till nod j endast användande noderna 1, …, k som mellannoder.

kortasteVägen(i,j,k+1) =
min(kortasteVägen(i,j,k), kortasteVägen(i,k+1,k)+ kortasteVägen(k+1,j,k))

där

kortasteVägen(i,j,0) = w(i,j)

Här kunde man nu ha löst detta top-down och tabellerat svar. Bottom-up blir dock mycket effektivare:

# Algoritm

Input:
V – mängden av noder numrerade 1, …, N
w(i,j) – vikt för båge från nod i till nod j

Output:
a(i,j) – 2D-array för avståndet från i till j (kortaste)

```
För alla bågar sätt a(i,j) = w(i,j)
Sätt a(i,i) = 0 för alla i
För k = 1 till N
        För i = 1 till N
                För j = 1 till N
                        a(i,j)=min(a(i,j),a(i,k)+a(k,j))
```

# Kappsäck

Det finns *n olika sorters* varor vars värde är $v_i$ och vikt är $w_i$. Av varje sorts vara finns det obegränsat antal. Hur ska vi packa en kappsäck som maximalt tar vikten W så att vi får med oss det maximala värdet?

Låt m(w) vara det maximala värdet vi kan få i i en kappsäck av storlek w. Då gäller:

$m(w) = \max(v_1+m(w-w_1), v_2+m(w-w_2),\ldots, v_n+m(w-w_n))$
där endast de fall då $w-w_i \geq 0$ är med. Vi har också att:
$m(0) = 0$.

Precis som i förra exemplet kan det göras rekursivt top-down med tabulering men blir effektivast bottom-up där vi bestämmer maximala värdet för en kappsäck med plats för 1, 2, …, W.

# Dijkstra

Även Dijkstras algoritm kan sägas vara dynamisk. Den sparar ju den just nu närmsta vägen till en nod och återanvänder detta resultat för att kunna beräkna just nu närmsta vägen till angränsande noder.

# Dynamiskt och bredden först

Inte bara rekursiva lösningar utan även när vi köar tillstånd för att köra bredden först kan vi också spara tillstånden så att vi inte köar samma tillstånd två gånger. Se t.ex. hissen:

```
private static int antalResorBredd(int n, int upp, int ned, int destination) {
    Queue<Tillstand> q = new ArrayDeque<>();
    int[] memo = new int[n+1];
    Tillstand t = new Tillstand(1, 0);
    while (t.position != destination||t.antalResor>30) {
        if (t.position + upp <= n && memo[t.position+upp] == 0) {
            q.offer(new Tillstand(t.position + upp, t.antalResor + 1));
            memo[t.position+upp]=1;
        }
        if (t.position - ned >= 1 && memo[t.position-ned] == 0) {
            q.offer(new Tillstand(t.position - ned, t.antalResor + 1));
            memo[t.position-ned]=1;
        }
        t = q.poll();
    }
    return t.antalResor;
}
```

```
private static class Tillstand {
    public int position, antalResor;
    public Tillstand(int p, int a) {
        position = p;
        antalResor = a;
    }
}
```

Observera att vi inte behöver spara några resultat eftersom om vi redan köat ett tillstånd har det som mest lika många resor. Detta räknas kanske inte som dynamisk programmering men är mycket närbesläktat.

# Hashtabell

Dynamisk programmering top down så behöver vi lagra resultat av beräkningar som vi senare snabbt vill kolla upp. Om resultaten kan namnges med ett index i en array blir detta mycket smidigt och enkelt, tex fibonacci, myntväxling, kappsäck.

Dock ibland kan dellösningarna inte enkelt namnges med ett index i en array. Då är ofta en hashtabell en ypperlig datastruktur för att lagra dellösningarna. Vi kan ha hur avancerade nycklar (key) som helst och ändå är det O(1) att ta ut resultatet (value) ur hash-tabellen.

Ett exempel på detta får vi i nästa ganska avancerade exempel men kom ihåg principen även för enklare problem.

# Handelsresandeproblemet

Givet n städer (1,…n) och avståndet mellan varje par av städer, w(i,j). Vilken väg ska en handelsresande välja för att minimera resvägen och besöka alla städer exakt en gång och börja och sluta i samma stad?

*Kan vi inte bara prova alla kombinationer? O(n!)*

# Held Karp

Algoritmen bygger på iden att varje delväg av hela den minimala vägen är själv den minimala vägen i den delmängden av städer den ingår i.

Kan implementeras dynamiskt top-down eller bottom-up

Blir $O(n^2 2^n)$ – inte lysande men bästa möjliga för att garanterat hitta bästa vägen.

# Top down

V - mängden av alla städer **inklusive** stad 1.

$S \subseteq V$ och $S \neq \emptyset$

Låt $j \in S$. Då är b(S,j) –

kortaste vägen som börjar i j går genom alla städer i V \ S (dvs resten av städerna) och sedan avslutar i stad 1.

Lösningen blir då: b({1},1)

**b(S,j)**

Om S=V returnera w(j,1)

Annars returnera $\min_{i \in V \setminus S} [b(S \cup \{i\},i)+w(j,i)]$

-för att det ska bli dynamiskt måste vi då tabulera b(S,j) så att vi inte räknar om samma värden flera gånger. Vi behöver alltså spara kortaste vägen för en given delmämgd S och en given startstad j. Detta gör vi lämpligen i en hashtabell.

Obs att \ är subtraktion i mängdläran

# Implementering utan tabulering, dvs n!

```
public static int minDistanceNaive(int[][] w){
    BitSet visited = new BitSet(w[0].length);
    visited.set(0);
    return minDistanceNaive(w,0, visited);
}
```

```
private static int minDistanceNaive(int[][] w,int currentCity, BitSet visited){
    if(visited.cardinality()==w[0].length){
        return w[currentCity][0];
    }else{
        int distance;
        int minDistance = Integer.MAX_VALUE;
        for(int i=1;i<w[0].length;i++){
            if(!visited.get(i)){
                visited.set(i);
                distance = w[currentCity][i]+minDistanceNaive(w,i,visited);
                if(distance<minDistance) minDistance=distance;
                visited.clear(i);
            }
        }
        return minDistance;
    }

}
```

# Implementering med tabulering $n^2 2^n$

```
public static int minDistanceDynamic(int[][] w){
    BitSet start = new BitSet(w[0].length);
    start.set(0);
    PartPath startPath = new PartPath(start,0);
    HashMap<PartPath,Integer> memo = new HashMap<>();
    return minDistanceDynamic(w,startPath,memo);
}
```

```
private static int minDistanceDynamic(int[][] w,PartPath currentPath,HashMap<PartPath,Integer>
memo){
    if(memo.containsKey(currentPath))
        return memo.get(currentPath);
    if(currentPath.visited.cardinality()==w[0].length){
        memo.put(currentPath,w[currentPath.current][0]);
        return w[currentPath.current][0];
    }else{
        int distance;
        int minDistance = Integer.MAX_VALUE;
        for(int i=1;i<w[0].length;i++){
            if(!currentPath.visited.get(i)){
                PartPath newPath = currentPath.add(i);
                distance = w[currentPath.current][i]+minDistanceDynamic(w,newPath,memo);
                if(distance<minDistance) minDistance=distance;
            }
        }
        memo.put(currentPath,minDistance);
        return minDistance;
    }
}
```

```java
private static class PartPath {
    private BitSet visited;
    private int current;

    public PartPath(BitSet s, int current) {
        this.visited = s;
        this.current = current;
    }

    public PartPath add(int newCurrent){
        BitSet s = (BitSet)visited.clone();
        s.set(newCurrent);
        return new PartPath(s,newCurrent);
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        PartPath partPath = (PartPath) o;
        return current == partPath.current &&
                visited.equals(partPath.visited);
    }

    @Override
    public int hashCode() {
        return Objects.hash(visited, current);
    }
}
```

# Bottom-up

n –antal städer och vi börjar och sluta i stad 1.

V - mängden av alla städer **exklusive** stad 1.

$S \subseteq V$ och $S \neq \emptyset$

Låt $j \in S$. Då är

a(S,j) – kortaste vägen som börjar i 1 passerar alla städer i S exakt en gång och slutar i j

Här bygger vi helt enkelt upp a(S,j) börjande nedifrån till dess att vi får a(V,j):

För alla j

    a({j},j) = w(1,j)

För s = 2 till n-1 //s är storleken på (ordningen av) S

    För alla subset S av storlek s

        För alla $j \in S$

            $a(S,j)=\min_{i \in S-\{j\}} [a(S\setminus\{j\},i)+w(i,j)]$

return $\min_{j \in V}[a(V,j)+w(j,1)]$

# Top down – variant med a(S,j)

Låt oss bestämma oss för att börja och sluta i stad 1.

V - mängden av alla städer **exklusive** stad 1.

$S \subseteq V$ och $S \neq \emptyset$

Låt $j \in S$. Då är

a(S,j) – kortaste vägen som börjar i 1 passerar alla städer i S exakt en gång och slutar i j

Lösningen blir då: $\min_{j \in V}[a(V,j)+w(j,1)]$

**a(S,j)**

Om S={j} returnera w(1,j)

Annars returnera $\min_{i \in S-\{j\}} [a(S\setminus\{j\},i)+w(i,j)]$

-för att det ska bli dynamiskt måste vi då tabulera a(S,j) så att vi inte räknar om samma värden flera gånger….

Obs att \ är subtraktion i mängdläran